

Updating the World Is Not Rebinding the Target: Temporal Referent Integrity in Tool-Using Agents

Anonymous submission

Abstract

Refreshing external state does not by itself authorize a tool-using agent to change the entity denoted by an earlier reference. We formalize this post-binding problem as *temporal referent integrity* (TRI), separating belief updates from referent transitions. On a frozen 160-task diagnostic, a Generic Structured Ledger reaches 64.4%/71.9% with Qwen3.5-122B/GLM-5.1, whereas pre-refresh Compile-then-act (CTA) reaches 95.0%/96.2%. Always-Lock and Always-Reevaluate each reach 60.0% and fail opposite modes. In a separately frozen 240-task replication, conditional on correct initial binding, Generic drifts on 43/72 and 38/80 opportunities; CTA and Lifecycle-Gated drift on none, and all 81 Generic drifts replay as wrong-entity SQLite writes. A post-primary DeepSeek replication finds 59/79 versus 0/70 Generic/CTA drifts. A typed Lifecycle reaches 98.1%/100.0% on the primary set, but its gate adds only 1.2–1.9 points over a matched free actor. Three blind annotators obtain Fleiss’ $\kappa = .708$ and 86% majority-gold agreement on 100 original/rewrite items. Public ToolSandbox, AppWorld, and τ^3 audits find almost no native strict opportunities. A post-hoc audit finds 3/6 GLM Generic violations in a frozen ToolSandbox-compatible intervention, while lower-intervention tool loops are null. TRI is a controlled, model- and controller-conditional diagnosis, not a prevalence or safety claim.

Introduction

Consider two requests issued before the same mailbox refresh:

- (1) “Choose the highest-priority unread email now. Refresh the mailbox, then reply to it.”
- (2) “Refresh the mailbox first. Then choose the highest-priority unread email and reply.”

Suppose email A is highest priority before refresh and email B afterward. The environment transition is identical, but the correct target differs. Request (1) establishes A as an identity commitment; request (2) deliberately leaves a description to be evaluated in the new state. Acting on B in (1) is not stale-state correction. It is a wrong-entity action.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

This distinction is easy to lose in LLM agent controllers. A controller may preserve the entire transcript, append the latest observation, and ask a downstream model to act. All facts remain available, yet the model can silently apply the old selector to the new state. Conversely, always retaining an old ID breaks genuinely dynamic references. Reliable tool use therefore needs *selective invariance*: world facts should update while some target denotations remain fixed, others are reevaluated, and invalid commitments follow an explicit fallback policy.

We call this property *temporal referent integrity* (TRI). TRI is not ordinary entity memory. Remembering an ID does not say whether that ID is a committed referent or merely the previous output of a dynamic selector. Nor does existence imply action validity: an email may remain present but cease to be replyable. Finally, the selector that established identity is not necessarily a persistent validity predicate. A branch chosen because it was default can stop being default while remaining the correct branch on which to run checks.

Figure 1 summarizes the problem and our controller. A Generic Structured Ledger records identity, facts, selector, and action preconditions, while leaving the actor to infer how those fields interact across refresh. Minimal Compile-then-act (CTA) first records binding time and identity. Its typed Lifecycle-Gated extension adds validity and fallback state; a deterministic gate enforces preserved identities while an actor resolves dynamic branches.

Our contributions are threefold:

- We formalize TRI as a post-binding authorization problem separating reference mode, bound identity, selector provenance, action-relative validity, and guarded fallback. Latest-state observations can be informationally insufficient; full history may contain the answer while still leaving the commitment computationally implicit and unenforced.
- We introduce frozen clustered diagnostics for language-template clusters, schema transfer under shared language templates, and guarded conditional commitments. We pre-specify cluster-resampled statistics and audit API failures, missing tasks, retries, and stage-wise errors.
- Across three model families, independent human rewrites, SQLite mutations, and public trace audits, component evidence most strongly supports pre-refresh com-

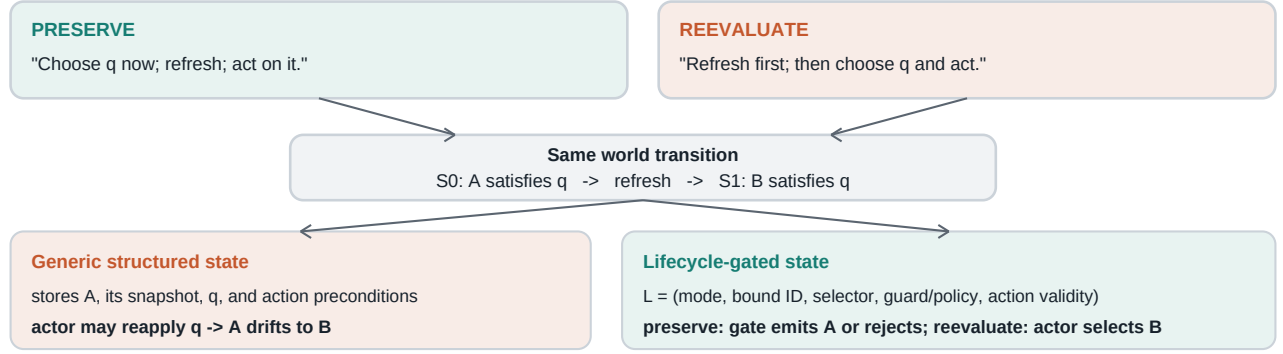


Figure 1: The same state transition requires different targets depending on discourse. Generic structured state preserves facts but leaves the interaction between identity and selector implicit. Lifecycle state makes the permitted denotation update executable.

mitment compilation. Minimal CTA has the highest or tied scalar point estimate; typed lifecycle state exposes a richer authorization contract, while negative stress tests localize its compilation and compositional costs.

Related Work

Reference, binding, and memory. Discourse theories distinguish referents from changing descriptions and salience (Kamp and Reyle 1993; Grosz, Joshi, and Weinstein 1995); coreference work recovers textual identity links (Lee et al. 2017; Joshi et al. 2020; Dasigi et al. 2019). Entity Binding Failures studies wrong initial tool arguments (Babu and Indukuri 2026); TRI instead conditions on correct $bind(r, S_0)$ and asks whether S_1 licenses a transition. Agent memory and entity tracking retain facts or traces (Tang et al. 2026; Packer et al. 2023; Sumers et al. 2024; Yao et al. 2023), but an ID alone does not state whether it is committed or a stale selector result. Evolving- environment benchmarks study belief and memory adaptation (Ji et al. 2026; Xu et al. 2026); TRI instead asks whether adaptation licenses changing a resolved action referent.

Binding Drift. The closest neighbor studies silent entity replacement in multi-step workflows and evaluates locking and re-verification (Indukuri 2026). TRI overlaps with its Preserve branch but also includes authorized Reevaluate cases: “choose now, refresh, act” and “refresh, then choose” require opposite targets under the same transition. We reproduce its deterministic offline harness, but its oracle re-verifier reads a gold target; its Entity Lock corresponds to our Always-Lock extreme. Its learned self/cross re-verifiers resolve the original step-1 referent and target initial binding or propagation; they do not expose a separate user-authorized deferred query. TRI therefore studies authorization of transition, not persistence alone. We do not claim an official LLM-sweep reproduction. In a post-primary 20-task author adaptation, the lock analogue scores 10/10 Preserve and 0/10 Reevaluate, while GLM re-verification reverses this to 0/10 and 10/10; Qwen re-verification is dominated by selector-grounding errors. Frozen CTA is the only tested policy nonzero on both

modes for both models (full table supplementary). Initial-binding correctness and post-update reevaluation authorization are orthogonal axes: Binding Drift targets the former, while TRI conditions on it to isolate the latter (matrix and audit details are supplementary).

Tool agents and executable constraints. Tool-use benchmarks cover APIs, web environments, and stateful interaction (Schick et al. 2023; Li et al. 2023; Qin et al. 2024; Patil et al. 2023; Liu et al. 2024; Zhou et al. 2024; Lu et al. 2025; Trivedi et al. 2024; Yao et al. 2024; Barres et al. 2025; Sierra Research 2026). Their gold does not label post-binding Preserve/Reevaluate opportunities. LedgerAgent and Bounded Autonomy add typed state or action contracts (Uddin et al. 2026; Sohail and Haider 2026); action validity still cannot determine which valid entity discourse authorized. Structured generation can enforce syntax (Beurer-Kellner, Fischer, and Vechev 2023; Willard and Louf 2023; Khattab et al. 2024), while our clustered minimal pairs follow diagnostic and contrast-set methodology (Ribeiro et al. 2020; Gardner et al. 2020).

Problem Formulation: Temporal Referent Integrity

Let H_t be interaction history, S_t world state, r an action-target reference, and q_r its selector. Referential control state is either an unresolved query $C_t(r) = U(q_r)$ or a bound commitment $C_t(r) = B(e)$. Language and history induce a transition authorization $\Gamma_{t+1}(r, e \rightarrow e')$; a world update alone does not:

$$\begin{aligned} S_t \neq S_{t+1} &\not\Rightarrow C_t(r) \neq C_{t+1}(r), \\ B(e) \rightarrow B(e'), e' \neq e &\Rightarrow \Gamma_{t+1}(r, e \rightarrow e'). \end{aligned} \quad (1)$$

Thus a selector can be evaluated to establish $B(q_r(S_0))$, preserved across refresh, or left as $U(q_r)$ for authorized evaluation in S_1 . Returning from $B(e)$ to query evaluation requires authorization; a refreshed selector winner is evidence about the world, not such authorization.

For execution, $V_a(e, S)$ denotes action-relative validity and π an invalid-target policy. Our lifecycle record

$L = (m, e, q, g, V_a, \pi)$ is one implementation of this state machine, not the definition of TRI. TRI-v3 uses $m \in \{\text{PRESERVE}, \text{REEVALUATE}\}$; TRI-v4 adds a conditional guard g .

For the scalar policies evaluated here, the execution branches are

$$\text{PRESERVE}, V_a(e, S_1) \Rightarrow e, \quad (2)$$

$$\text{PRESERVE}, \neg V_a(e, S_1), \pi = \text{REJECT} \Rightarrow \perp, \quad (3)$$

$$\text{REEVALUATE} \Rightarrow q(S_1), \quad (4)$$

$$\text{CONDITIONAL}, g(e, q, S_1) \Rightarrow e, \quad (5)$$

$$\text{CONDITIONAL}, \neg g(e, q, S_1) \Rightarrow q(S_1). \quad (6)$$

An invalid preserved entity remains $B(e)$ even when execution returns $\perp$. Likewise, q establishes identity under Preserve but remains unresolved under Reevaluate; action validity never authorizes substituting a different valid entity. Full history may contain all necessary facts while leaving this relationship computationally implicit, and correct inference still requires enforcement at the mutation boundary. TRI-v4 distinguishes $g_{\text{act}} = V_a(e, S_1)$ from $g_{\text{sel}} = [q(S_1) = e]$.

Proposition (mode-blind policy impossibility). Consider a Preserve/Reevaluate minimal pair with identical S_0, S_1, q , and a , where $e = q(S_0)$, $e' = q(S_1)$, and $e \neq e'$. The authorized targets are e and e' , respectively. Any deterministic post-refresh policy of the form $f(e, q, S_1, a)$, containing neither the original history nor a compiled transition authorization, receives the same inputs for both tasks and must return the same target; it is therefore wrong on at least one member. Always-Lock and Always-Reevaluate instantiate the two complementary extremes measured in our policy controls. This establishes the need for a discourse-sensitive transition decision, not the necessity of our particular serialization.

Authorization invariant (conditional guarantee). If L is correct, IDs are stable, selector/guard evaluation is correct, and the executor follows the branches above, every emitted target is referentially authorized; enforcing V_a additionally makes each executed mutation action-valid. The guarantee ends at compilation: a wrong but valid ID, identity migration, or an incomplete action schema can still pass.

Outcomes. We separately report exact target/final state, wrong-entity writes, invalid attempts, and unnecessary rejection; always rejecting avoids writes but fails every actionable task.

Method: Pre-Refresh Commitment Compilation

The controller family first compiles (u, S_0) before refresh. Minimal CTA emits binding time, selector, and a bound ID. The typed Lifecycle extension additionally emits

```
reference_mode; bound_target_id;
selector; invalidity_policy.
```

Both compilers use discourse order: “identify, refresh, act on it” preserves identity even without words such as “same,” while “refresh before deciding” reevaluates. It resolves a concrete bound ID only for preserved references.

The executor then refreshes the environment. If the compiled mode is PRESERVE, a deterministic gate checks the bound entity against action preconditions. The gate emits the bound ID when valid, returns $\perp$ under a reject policy, or invokes selector resolution only when the compiled fallback authorizes it. If the mode is REEVALUATE, the LLM actor selects from S_1 . Thus a preserved commitment is a controller constraint, not advice that a second model may reinterpret. The preserve branch also needs one model call rather than two.

TRI-v4 generalizes the record with `guard_type` and `fallback_policy`. The action-validity guard can be enforced deterministically. A selector-match guard is evaluated by the actor because the current prototype compiles natural language selectors but does not execute them as a general symbolic query language. This design choice exposes selector grounding as a measurable stage rather than using gold selector fields.

The complete controller procedure and model-facing interfaces are provided in the supplement. Gold targets and benchmark selector fields are never provided to the compiler or actor.

Information fairness. Compilers see the instruction, S_0 , action schema, and planned refresh, never gold targets or generator selector fields. Generic retains the instruction, selected ID, snapshot, selector, action, and preconditions for its second-call actor. CTA, Lifecycle-free, and Generic each use two calls; Lifecycle-Gated skips the actor for a valid Preserve branch. Full prompts, interfaces, and executor-level oracle checks are supplementary.

Experimental Setup

Controlled task construction. Each domain specification defines an entity table, a natural-language selector, its ranking or Boolean criterion, an action, and explicit action preconditions. The generator first evaluates the selector on S_0 to obtain e_0 , applies a controlled transition, and evaluates it on S_1 to obtain e_1 . A language template determines whether the action target is $e_0, e_1, \perp$, or a conditionally guarded choice. Paired tasks share the domain, both states, selector, action, and schema; only discourse order and anaphoric language change. This prevents domain difficulty or target frequency from explaining the preserve/reevaluate contrast.

The five transition types serve different diagnostic roles. *Flip* changes which valid entity wins the selector. *Stable* leaves the winner unchanged and detects controllers that overreact to any refresh. *Remove* deletes the pre-refresh entity. *Invalidate* keeps its ID present while changing an action-precondition field, separating existence from validity. *Name collision* gives another entity the same display label while changing the selector winner, testing whether controllers preserve stable identity rather than surface aliases. Across anchored and dynamic language, these updates create matched cases where naive “always old” and “always new” policies have complementary failures.

Blind human construct validation. One independent volunteer naturally rewrote 50 sampled scalar instructions.

Controller	Qwen3.5	GLM-5.1
Generic free actor	64.4	71.9
Generic + reference mode	75.0	75.0
Generic + validity gate	65.0	73.1
Untyped pre-refresh plan	81.2	70.6
Exact compile then act	95.0	96.2
Lifecycle free actor	96.9	98.1
Lifecycle + gate	98.1	100.0

Table 1: Exact target accuracy (%) on the same 160-task v3 language set. The post-freeze Generic+reference-mode ablation adds only the mode field; Lifecycle-free is statistically tied with historical Compile-then-act, and the gate adds only 1.2–1.9 points.

Three other volunteers, uninvolved in TRI design, independently labeled 100 randomized original/rewrite items using opaque IDs without condition metadata or gold answers. They selected an entity, REJECT, or CLARIFY; confidence was optional. We report majority–gold alignment, unanimous and pairwise agreement, Fleiss’ κ , nominal Krippendorff’s α , and a consensus-only sensitivity analysis. Conditional, collection, and nested cases were not included in this human study.

Stage and write-consequence reports use a frozen mechanism-oriented error taxonomy, detailed in the supplement, that separates temporal rebinding, premature binding, invalid processing, unnecessary rejection, and selector-grounding errors.

Strong baselines and component audit. Table 1 reports the frozen v3 primary inventory. Generic validity gating, an untyped pre-refresh plan, Lifecycle-free execution, and exact historical Compile-then-act were post-primary audits; they do not replace the pre-specified Generic-versus-Lifecycle-Gated estimand. They do, however, determine how narrowly that comparison can be interpreted.

TRI-v3 language clusters. The primary test contains 160 tasks formed from 20 frozen language-template clusters, four reference styles, and eight domains per template. Within each style, flip, stable, remove, invalidate, and name-collision updates are balanced. The pre-specified statistic resamples whole template clusters with 10,000 bootstrap samples; task-level McNemar is secondary. Before any model call, we froze the data, runner, hashes, inference settings, and rule that prompts would not change after output inspection.

Schema transfer under shared language templates and guarded policies. An 80-task transfer set crosses the same 20 template clusters with four new domains: projects, expenses, inventory, and cloud deployments. These introduce new IDs, fields, selectors, actions, and action preconditions. TRI-v4 separately freezes 40 conditional tasks, ten template clusters, and the two guards g_{act} and g_{sel} . Every guard–update–domain combination occurs once. TRI-v4 is a secondary extension, not part of the pre-specified v3 primary comparison.

Direct-neighbor baseline. Generic Structured Ledger uses two calls. Its compiler stores the initial selected ID, complete entity snapshot, selector, requested action, and action preconditions, but omits reference mode and invalidity policy. Its actor still receives the original instruction, ledger, S_1 , and action schema. It is therefore not an information-deletion baseline: the comparison asks whether structured facts suffice without discourse commitment semantics.

The historical Compile-then-act prompt is rerun unchanged on the v3 language set (Table 1). Its cluster interval overlaps lifecycle-free execution for both models, consistent with pre-refresh compilation, rather than tuple serialization or deterministic gating alone, being the leading explanation. Because it emits binding time and a bound ID, it also serves as the simple mode-classifier-plus-ID-lock analogue for scalar cases. The typed contract contributes an auditable interface for validity and fallback; the gate is a small enforcement layer, not the principal accuracy mechanism. A matched free-form pre-refresh planner reaches only 81.2% for Qwen and 70.6% for GLM, so moving computation before refresh is not alone sufficient; nevertheless, the strong historical predecessor prevents us from claiming that the full tuple is empirically necessary for scalar tasks.

For a direct policy contrast, we also evaluate two deterministic extremes without model calls. Always-Lock + validity retains the S_0 ID and rejects it only when the refreshed action preconditions fail; Always-Reevaluate discards the binding and applies q to S_1 . These are conceptual control-lines rather than implementations of the Binding Drift baselines. On the frozen 160-task inventory both reach 96/160 (60.0%), but their mode slices are complementary: Always-Lock reaches 80/80 anchored and 16/80 dynamic, whereas Always-Reevaluate reaches 16/80 anchored and 80/80 dynamic. The contrast is a diagnostic of selective authorization, not evidence that either unconditional policy is a competitive agent controller.

Model-facing SQLite trajectories. We freeze a 40-task subset covering all 20 templates, eight domains, both modes, and all update types. The runtime queries an in-memory SQLite database, gives returned rows to the compiler, replaces the database state during refresh, obtains a target from the gate or actor, and executes UPDATE through a mutation tool that enforces action preconditions. We inspect final database state, wrong writes, invalid attempts, and collateral modifications. Query and required refresh order are controller-orchestrated; this is a focused execution test, not a fully autonomous external benchmark.

Models, inference, and auditing. We use Qwen3.5-122B-A10B and GLM-5.1 through the same OpenAI-compatible API at temperature zero, with thinking disabled and a 1,200-token output cap. A post-primary robustness replication uses DeepSeek-V4-Pro with the same settings. Balanced smoke tests gate each expansion. Final reports verify expected task IDs, duplicates, status, request attempts, and retries. All reported v3/v4 full files are complete and contain zero API errors and zero retries. API failures would be reported separately rather than interpreted as model behavior.

Statistical estimands. The primary estimand is Lifecycle-Gated minus Generic Structured Ledger accuracy on the Qwen language-cluster set. We resample its 20 complete template clusters rather than treating 160 templated rows as independent. The same cluster bootstrap is reported for replication and transfer sets; each interval uses 10,000 samples and a fixed recorded seed. Template-macro and task accuracy coincide in the balanced sets, but both are retained in machine-readable reports. Exact paired McNemar tests use task-level discordance and are secondary. Wilson intervals describe individual proportions. Slices by binding, explicitness, update, and domain are mechanism analyses rather than additional confirmatory hypotheses; we do not use them to select the primary claim.

Reproducible run discipline. Frozen data, protocols, runners, and smoke subsets have recorded SHA-256 hashes. Reports retain raw outputs, attempts, retries, stage errors, and SQLite traces; interrupted rows are auditable by ID. The anonymous artifact maps every main-table value to a raw run and report script, includes the human-analysis code and de-identified responses, and excludes API keys and private coordination files. Post-primary audits are labeled as such.

Results

Primary and Transfer Results

Structured facts are not a commitment. Table 2 shows the same mechanism across models. On the primary language-cluster test, Generic Structured Ledger reaches 64.4% for Qwen and 71.9% for GLM, despite carrying identity, facts, selector, preconditions, and the original instruction. Lifecycle-Gated reaches 98.1% and 100.0%. Paired discordance is 3 generic-only versus 57 lifecycle-only for Qwen ($p = 6.25 \times 10^{-14}$), and 0 versus 45 for GLM ($p = 5.68 \times 10^{-14}$). Qwen improves from 33.8% to 100.0% on anchored references while remaining comparable on dynamic references (95.0% versus 96.2%). GLM improves from 56.2% to 100.0% anchored and from 87.5% to 100.0% dynamic. The gain is therefore not explained by always preserving old IDs.

Because the Lifecycle compiler receives explicit reference-mode and invalidity semantics, this comparison does not by itself identify the causal contribution of serialization, prompt supervision, and the deterministic gate separately. The historical compile-then-act results in Table 1 provide a strong neighboring comparison and motivate the narrower interpretation: the gain is primarily associated with explicit pre-refresh compilation, while lifecycle state makes its authorization and validity decisions auditable and enforceable.

The post-primary component audit sharpens this conclusion. A validity-only gate changes Generic accuracy by only +0.6 points for Qwen and +1.2 for GLM. Lifecycle state with a free actor reaches 96.9% and 98.1%; deterministic enforcement adds only +1.2 and +1.9 points. The new Generic+reference-mode ablation reaches 75.0% for both models, with paired cluster differences of +10.6 [+5.0, +16.9] and +3.1 [-2.5, +9.4] points. Thus explicit mode classification is useful but model-dependent and insufficient to explain CTA’s full gain. The call-matched untyped

plan reaches 81.2% and 70.6%, 15.6 and 27.5 points below Lifecycle-free with cluster intervals excluding zero, showing that pre-refresh timing alone is insufficient. Exact historical CTA reaches 95.0% and 96.2%, tied with Lifecycle-free; the audit therefore supports commitment compilation as the main mechanism, with typed lifecycle state adding auditable validity and fallback rather than explaining the full accuracy gap.

The deterministic policy controls further separate TRI from unconditional persistence. Always-Lock + validity and Always-Reevaluate each reach only 60.0% overall. Their equal totals hide opposite errors: lock is 100.0% on anchored and 20.0% on dynamic references, while reevaluation is 20.0% anchored and 100.0% dynamic. TRI’s empirical target is the discourse-conditioned choice between these transitions, not locking by default.

Because the aggregate includes a normative invalid-target policy, we also separate its 128 actionable core items: Generic/Historical-CTA/Gated reach 74.2/98.4/97.7% for Qwen and 72.7/99.2/100.0% for GLM. The complete 32-item policy slice is supplementary; this post-primary sensitivity does not replace pre-specified total accuracy.

The strongest generic failures occur when the bound entity remains valid but no longer wins the selector: GLM obtains 18.8% on anchored flips and 25.0% on name collisions, while the lifecycle gate is perfect. Implicit commitments are also harder for generic state (42.5% anchored for GLM) than explicit ones (70.0%). A field-corrected stage audit is important here: Generic stores the initial target as `selected_entity_id`, not the lifecycle-only `bound_target_id`. Conditioning on the former being correct, Qwen drifts to the refreshed leader on 15/16 anchored flips and 14/16 name collisions; GLM does so on 3/16 and 7/16, respectively, while stable controls show 0 drift. These patterns match the predicted distinction between storing evidence and enforcing what that evidence means for denotation.

A larger frozen replication confirms conditional drift.

After selecting the mechanism, we froze 240 core-only tasks over ten unseen schemas and four independently parameterized states per schema. All old targets remain present and action-valid; the 40 state instances are the resampling units. Generic binds 107/120 Qwen and 120/120 GLM anchored targets correctly. Within the resulting flip/name-collision opportunities it then drifts to the refreshed winner 43/72 times (59.7%, state-cluster 95% CI [46.5, 72.4]) and 38/80 (47.5%, [35.0, 61.3]); stable final-target errors are 4/40 and 2/40. Exact CTA and Gated have zero such drift after a correct binding. Overall CTA improves over Generic by 23.3 points [16.2, 30.4] and 24.2 [19.6, 28.7]. Qwen’s lower dynamic accuracy exposes compound-selector grounding rather than temporal drift. In a later matched DeepSeek run on all 240 frozen tasks, Generic reaches 73.8% and drifts on 59/79 core opportunities, whereas CTA reaches 91.2%, drifts on 0/70, and improves accuracy by 17.5 points [10.8, 23.3]. SQLite replay turns all 59 DeepSeek Generic core drifts into wrong-entity writes; CTA still makes 17 non-core wrong writes from mode, initial-ID, or selector errors. Thus the replica-

Benchmark	Model	Generic E2E	CTA E2E	Lifecycle E2E	Δ	Cluster 95% CI
Language clusters (primary)	Qwen3.5	64.4	95.0	98.1	+33.8	[+18.1, +50.0]
Language clusters	GLM-5.1	71.9	96.2	100.0	+28.1	[+18.1, +38.1]
Core replication (new states)	Qwen3.5	47.5	70.8	71.2	+23.8	[+16.2, +31.2]
Core replication (new states)	GLM-5.1	70.0	94.2	97.1	+27.1	[+22.1, +32.1]
Schema transfer (shared templates)	Qwen3.5	46.2	—	82.5	+36.2	[+25.0, +47.5]
Model-facing SQLite	Qwen3.5	67.5	—	100.0	+32.5	[+15.0, +50.0]
Model-facing SQLite	GLM-5.1	65.0	—	100.0	+35.0	[+17.5, +52.5]
Guarded conditional policies	Qwen3.5	52.5	—	85.0	+32.5	[+15.0, +52.5]

Table 2: End-to-end (E2E) authorized-target accuracy (%); SQLite rows instead use final database-state success. These totals are not initial-binding accuracy or conditional TRI. CTA is shown where run on the identical inventory; Δ is Lifecycle/Guarded minus Generic. Intervals resample complete template or state-instance clusters.

tion strengthens the conditional behavioral diagnosis without converting the synthetic benchmark into a prevalence or general-safety estimate.

Leave-one-group-out sensitivity keeps the CTA–Generic difference positive after excluding any one of ten domains or any one of sixteen template families. Across Qwen, GLM, and DeepSeek, the domain-exclusion ranges are 18.5–25.5, 23.1–25.9, and 15.7–22.2 points; template-exclusion ranges are 20.4–26.3, 20.8–26.1, and 14.2–19.6. No single domain or template explains the gain.

We keep four denominators distinct: E2E target accuracy covers all 240 rows; initial binding covers 120 anchored rows; conditional TRI covers only correct bindings with a valid old target and changed winner; wrong-write rate counts every executed mutation to a non-gold entity. The unified mechanism table is supplementary.

Matched full-history baselines do not remove substitution. To test whether the Generic ledger itself induces failure, we froze two additional v7 baselines for all three models. An ordinary two-turn agent retaining the complete transcript obtains 63.3/67.1/68.8% for Qwen/GLM/DeepSeek. A strong one-shot post-refresh re-resolution prompt that explicitly asks whether to preserve, reevaluate, or reject improves this to 69.6/80.8/75.8%, but still substitutes the refreshed winner on 56/38/42 of 80 anchored changed-winner rows. CTA obtains 70.8/94.2/91.2%; versus the strong prompt its paired differences are +1.2 [−6.7, 9.2], +13.3 [8.8, 17.9], and +15.4 [9.2, 21.7] points. Thus CTA is tied on Qwen overall accuracy, not universally superior. Because full-history baselines expose no separately verified pre-refresh binding, their substitutions are unconditional and are not counted as conditional TRI. The result supports explicit, executable commitment over history retention or a late semantic re-resolution decision, while preserving that denominator distinction. It is a practical nearest-neighbor approximation, not an official Binding Drift self/cross reproduction.

Human judgments support the core transition, with a fallback boundary. Across 100 blinded items, Fleiss’ $\kappa = .708$, Krippendorff’s $\alpha = .709$, and majority–gold agreement is 86% (91.5% among 94 determinate items); original and rewritten language obtain 84% and 88%. Using the 46 determinate original-item majorities as alternative gold, Generic/CTA score 69.6/91.3% for Qwen and

73.9/89.1% for GLM; on the 35 unanimous-gold items, they score 85.7/100.0% and 88.6/97.1%. On 50 frozen volunteer rewrites, Generic/CTA/Lifecycle-free/Gated reach 60/90/88/90% for Qwen and 74/98/92/94% for GLM; against 48 rewrite majorities, CTA reaches 89.6/93.8%. Thus the compiled-method gap survives human-majority and high-consensus scoring. Action-invalid REJECT obtains only 55% majority–gold and 25% unanimity, so Preserve/Reevaluate is the human-supported core and Reject remains a normative action policy.

External and Compositional Boundaries

The external and compositional evaluations do not show a stable extension of the scalar result. In the 24-task ToolSandbox-style pilot, Generic, Lifecycle, Gated, and Untyped obtain 79.2/87.5/91.7/91.7% for GLM and 91.7/83.3/91.7/79.2% for Qwen, respectively. Gated permits one GLM and two Qwen wrong writes. We therefore treat this pilot as an external validity boundary check: the controller advantage is not stable once tool choice and upstream compilation are exposed. A post-hoc strict trace audit nevertheless finds a benchmark-compatible positive instance: conditional on a correct compiled ID, a preserved valid old target, and a Flip, GLM Generic writes the refreshed winner in 3/6 opportunities versus 0/2 Stable controls; GLM Lifecycle is 0/5. Qwen Generic is 0/6, while its free Lifecycle actor is 2/6 and deterministic gate replay removes both. These custom tasks use ToolSandbox’s native reminder database and search/modify tools, but are neither an official score nor prevalence evidence.

In a separate 96-task ToolSandbox-style extension with auditable binding, Qwen/GLM full-history agents and matched Generic Ledgers produce zero conditional TRI in 70/73 and 64/87 eligible opportunities. Their 6/13/5/4 wrong writes are initial selector or tool-order errors. This negative boundary shows that the controlled result is model- and controller-dependent, not universal.

Audits of unmodified ToolSandbox (129 families), AppWorld (244 families), and τ^3 (2,449 tasks; 10,832 released trajectories) find zero native strict opportunities and a few near-matches. One AppWorld Todoist family naturally preserves task IDs after reassignment makes them fail the original selector; 16 correct operations are followed by 16 same-ID comments. These audits support the task structure and

a coverage gap, not prevalence; checklist and candidate IDs are supplementary.

In a two-cluster custom AppWorld study, conditional TRI is 0/24 with observable binding and 0/28 after removing TRI prompt language; two wrong writes are pre-binding order errors. This is not an AppWorld leaderboard score.

A two-refresh Qwen stress test favors Generic over scalar Lifecycle (32/40 vs. 28/40; -10.0 points, cluster CI $[-35.0, 12.5]$). Role indexing improves Qwen from 35/40 to 39/40, but recovered GLM ties at 40/40. The claim therefore remains limited to controller-orchestrated, single-refresh, scalar mutations.

Error and Safety Analysis

Stage analysis localizes the remaining error. Primary lifecycle compilation is correct; remaining Qwen errors occur in the dynamic actor. Under schema transfer, mode remains 100% but anchored bound-ID accuracy falls to 55.0/50.0% for explicit/implicit language, concentrated in compound selectors. The gate rejects ineligible IDs, showing that TRI semantics and selector grounding are distinct problems.

Transfer and real write consequences. On new schemas, lifecycle improves Qwen from 46.2% to 82.5%. In model-facing SQLite, Generic gives Qwen/GLM 27/40 and 26/40 final states with 13/8 wrong writes. Given correct initial IDs, it writes the refreshed winner in 8/8 and 6/8 valid flip/name-collision opportunities versus 0/4 Stable controls; remaining wrong writes test reject policy. Lifecycle-Gated reaches 40/40 for both models with no wrong or collateral write. A validity-only gate still leaves all 13 Qwen wrong writes because replacements are action-valid; this is a contract result, not general tool safety.

Discussion

Commitments benefit from explicit controller state. Generic carries enough evidence, but leaves unstable whether a selector is still executable or only provenance for a committed ID. Pre-refresh compilation makes that distinction auditable.

Gates move, rather than erase, the reliability bottleneck. A gate cannot repair a wrong mode, ID, or guard. It enforces a correct contract, sometimes turning grounding errors into rejection. CTA/Gated produce no core TRI write after correct binding but retain non-core selector errors; enforcement is not a general write-safety guarantee.

Boundary results constrain the design claim. ToolSandbox and multi-referent results show that gates cannot repair bad compilation and scalar contracts do not compose automatically across roles. In the supported setting, gating also reduces primary requests from 320 to 240 per model.

Limitations

Scope and construct validity. TRI is a controlled diagnostic, not a prevalence estimate. It simplifies ambiguity, repair, partial observability, and identity migration. Human validation covers scalar semantics; low agreement on invalid-target rejection limits fallback claims. Composition remains open.

Causal attribution and external validity. Temperature-zero runs do not characterize execution variance or all model families. Replications remain synthetic. We reproduced Binding Drift’s deterministic assertions, but not its LLM sweep; its oracle re-verifier reads gold and Entity Lock matches Always-Lock. The ToolSandbox-compatible positive audit is post-hoc and small, lower-intervention loops are null, and no external study estimates real-traffic prevalence. Opportunity coverage and controller state affect observability.

Conclusion

World updates need not rebind targets. Controlled replications, matched full-history baselines, and SQLite writes support one narrow design principle: decide whether a description has become an identity commitment before incorporating later observations. CTA has the highest or tied scalar point estimate; typed state adds policy but also compilation errors, and gates enforce only correct contracts. External evidence is mixed, while public audits expose few native strict opportunities. TRI is therefore a controller- and model-conditional mechanism diagnosis, not a prevalence estimate, universal LLM failure, or autonomous-agent safety guarantee.

References

- Babu, R. S.; and Indukuri, S. 2026. Entity Binding Failures in Tool-Augmented Agents. arXiv:2606.30531.
- Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2025. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982.
- Beurer-Kellner, L.; Fischer, M.; and Vechev, M. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1946–1965.
- Dasigi, P.; Liu, N. F.; Marasović, A.; Smith, N. A.; and Gardner, M. 2019. Quoref: A Reading Comprehension Dataset with Questions Requiring Coreferential Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 5925–5932.
- Gardner, M.; Artzi, Y.; Basmova, V.; Berant, J.; Bogin, B.; Chen, S.; Dasigi, P.; Dua, D.; et al. 2020. Evaluating Models’ Local Decision Boundaries via Contrast Sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1307–1323.
- Grosz, B. J.; Joshi, A. K.; and Weinstein, S. 1995. Centering: A Framework for Modeling the Local Coherence of Discourse. *Computational Linguistics*, 21(2): 203–225.
- Indukuri, S. 2026. Binding Drift in Multi-Step Tool-Augmented Agents. GitHub repository.
- Ji, H.; Xiong, K.; Han, S.; Xia, P.; Qiu, S.; Zhou, Y.; Liu, J.; Li, J.; Li, B.; Zheng, Z.; Xie, C.; and Yao, H. 2026. ClawArena: Benchmarking AI Agents in Evolving Information Environments. arXiv:2604.04202.
- Joshi, M.; Chen, D.; Liu, Y.; Weld, D. S.; Zettlemoyer, L.; and Levy, O. 2020. SpanBERT: Improving Pre-training by Representing and Predicting Spans. *Transactions of the Association for Computational Linguistics*, 8: 64–77.
- Kamp, H.; and Reyle, U. 1993. *From Discourse to Logic: Introduction to Modeltheoretic Semantics of Natural Language, Formal Logic and Discourse Representation Theory*. Kluwer Academic Publishers.
- Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*.
- Lee, K.; He, L.; Lewis, M.; and Zettlemoyer, L. 2017. End-to-end Neural Coreference Resolution. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 188–197.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.
- Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.
- Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183.
- Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.
- Ribeiro, M. T.; Wu, T.; Guestrin, C.; and Singh, S. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–4912.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
- Sierra Research. 2026. τ^3 -Bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. GitHub repository.
- Sohail, S.; and Haider, G. 2026. Bounded Autonomy for Enterprise AI: Typed Action Contracts and Consumer-Side Execution. arXiv:2604.14723.
- Sumers, T. R.; Yao, S.; Narasimhan, K.; and Griffiths, T. L. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*.
- Tang, Z.; Zhao, Q.; Franco, G.; Wijaya, D.; Mueller, A.; Schuster, S.; and Kim, N. 2026. Do Language Models Track Entities Across State Changes? In *International Conference on Machine Learning*. ArXiv:2605.30233.
- Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Li, V. D.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. arXiv:2407.18901.
- Uddin, M. N.; Saeidi, A.; Blanco, E.; and Baral, C. 2026. Ledger-Agent: Structured State for Policy-Adherent Tool-Calling Agents. arXiv:2606.20529.
- Willard, B. T.; and Louf, R. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702.
- Xu, J.; Li, Q.; Wu, J.; Lan, Y.; Li, S.; Zhou, H.; Jiang, B.; Wang, L.; Wang, J.; Luu, A. T.; Xiong, C.; Park, H. W.; Hooi, B.; and Hu, Z. 2026. EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. arXiv:2606.13681.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. Taubench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
- Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.